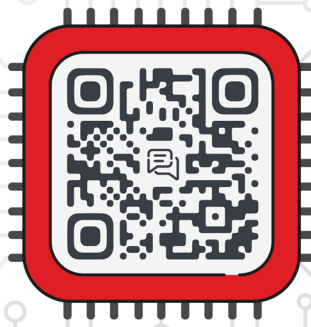




9



7



1050 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

السجلات struct

د. مدحت الصوص

د. فراس ضعيف

برمجة ا

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 18/1/2026

تعريف السجلات struct

السجلات (Struct) هي طريقة لتجميع مجموعة من البيانات مختلفة النوع ضمن متحول واحد، وهذا ما يميزها عن المصفوفات التي تستخدم لتجميع عناصر من النوع نفسه فقط. لتعريف هذا النمط المخصص في لغة C++ نكتب خارج الدالة main الكلمة المفتاحية struct، والتي تدل على إنشاء نوع بيانات جديد بواسطة المستخدم:

```
struct Student {
    string name;
    int ID;
};
```

يسمى student اسم السجل بينما تمثل المتحولات name و ID الحقول (أو عناصر السجل)

يمكن تعريف متحولات من نوع السجل بطريقتين:

• الطريقة الأولى: تعريف المتحول مباشرة بعد انتهاء تعريف السجل (قبل الفاصلة المنقوطة):

```
struct Student {
    string name;
    int ID;
} s1;
```

• الطريقة الثانية: تعريف المتحول بنفس طريقة تعريف أي متحول عادي:

```
Student s1;
```

من الشائع تعريف ال struct خارج الدالة main حتى يتمكن من استخدامه في أي تابع داخل البرنامج.



```
#include <iostream>
using namespace std;
struct Student {
    string name;
    int ID;
};

int main() {
    Student s1;
    s1.ID = 5;
    s1.name = "Ahmad";
    return 0;
}
```



نسمي المتحولات name و ID حقول السجل، ويمكن الوصول إليها باستخدام النقطة (.) كما يلي:

s1.name | s1.ID

فعند كتابة: s1.name نحن نتعامل مع متحول من النوع int

وعند كتابة: s1.ID فنحن نتعامل مع متحول من النوع string

توضيح مفهوم السجل

يمكننا تشبيه اسم السجل في لغة C++ بالأنماط الأساسية المعروفة مثل int و double و string، ولكن الفرق أن السجل هو نمط بيانات نقوم نحن بتعريفه.

- فعندما نكتب int x فإن int هو نوع بيانات جاهز في اللغة، و x متحول من هذا النوع.
- وعندما نكتب Student s1 فإن Student هو نوع بيانات قمنا بتعريفه باستخدام struct، و s1 متحول من هذا النوع.

int نوع بيانات جاهز من اللغة.

Student نوع بيانات قمنا نحن بإنشائه.

وبذلك نتعامل مع السجل تمامًا كما نتعامل مع أي نوع بيانات آخر، فنستطيع تعريف متحولات منه، وتمريرها إلى توابع، وتخزينها ضمن مصفوفات، والتعامل معها داخل البرنامج.

كما يمكن تشبيه السجل بعلبة كبيرة تحتوي مجموعة من العلب الصغيرة حيث تمثل العلبة الكبيرة متحول السجل (مثل s1)، وتمثل العلب الصغيرة الحقول الموجودة داخله مثل name و ID.

وعند كتابة: s1.name فإننا نتعامل مع الحقل name الموجود داخل المتحول s1.

وبذلك يمكننا الفهم بأن السجل ليس مجرد مجموعة متحولات، بل هو نوع بيانات جديد يمكن استخدامه في البرنامج كما نستخدم الأنماط الأساسية.

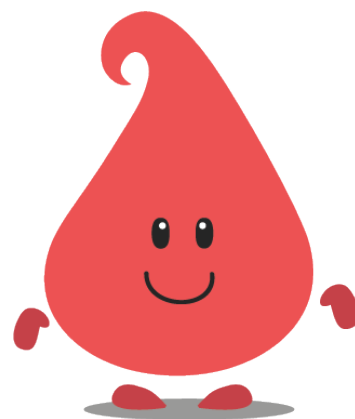
ملاحظات مهمة:

- لا يمكن كتابة Student.name لأن Student هو اسم نوع وليس متحولاً في الذاكرة لذلك لا يمكن إسناد قيم له مباشرة
- لا يقوم الـ struct بحجز مساحة في الذاكرة إلا عند تعريف متحول منه.

تمرير struct إلى تابع

يمكن تمرير متحول من نوع struct إلى تابع كما يلي:

```
#include <iostream>
using namespace std;
struct Student {
    string name;
    int ID;
};
void print(Student x) {
    cout << "Name: " << x.name << endl;
    cout << "ID: " << x.ID << endl;
}
int main() {
    Student s1;
    cin >> s1.name;
    cin >> s1.ID;
    print(s1);
    return 0;
}
```



لماذا لا يمكن استخدام الإدخال مباشرة بالشكل cin>>s1؟

لا يمكن إدخال القيم إلى متحول من نوع struct دفعة واحدة باستخدام cin لأن الإدخال المباشر يكون للأنماط البسيطة فقط مثل: int, double, string. أما الـ struct فهو نمط مركّب يحتوي عدة حقول، لذلك يتم إدخال القيم لكل حقل على حدة، تمامًا كما نفعل عند إدخال عناصر المصفوفة عنصراً عنصراً.

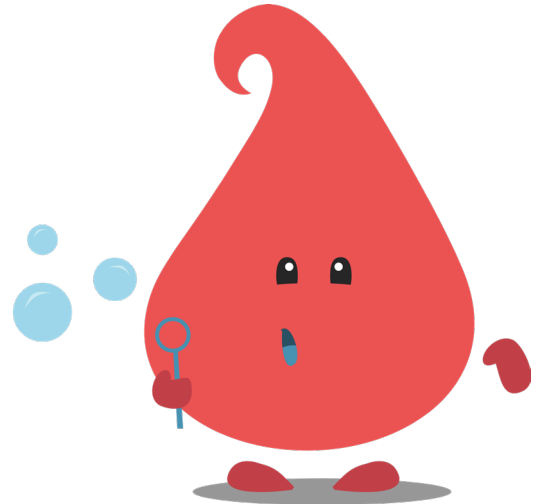
أهمية استخدام struct في تابع

يساعد استخدام السجلات (Struct) على تنظيم البيانات داخل البرنامج، إذ يمكن التعامل مع مجموعة من القيم المرتبطة معاً على أنها كتلة واحدة. كما يمكن إرسال متحول من نوع struct إلى تابع واحد بدلاً من إرسال كل قيمة على حدة، مما يجعل الكود أكثر وضوحاً وتنظيماً.

سجل داخل سجل آخر struct داخل struct

تسمح لنا C++ بتعريف سجل داخل سجل آخر، وذلك بهدف تنظيم البيانات المرتبطة ببعضها بطريقة منطقية وواضحة. فعندما يحتوي كيان معين على بيانات مركبة يمكننا تعريف سجل مستقل ثم استخدامه كحقل داخل سجل آخر. مثال:

```
#include <iostream>
using namespace std;
struct Date {
    int day, month, year;
};
struct Student {
    string name;
    int ID;
    Date birthday;
};
int main() {
    Student s1;
    cin >> s1.name;
    cin >> s1.birthday.day;
    cin >> s1.birthday.month;
    cin >> s1.birthday.year;
    return 0;
}
```



في المثال السابق قمنا بتعريف سجل باسم Date يحتوي معلومات التاريخ (اليوم، الشهر، السنة)، ثم استخدمنا هذا السجل كحقل داخل سجل الطالب Student تحت الاسم birthday وبذلك أصبح السجل Student يحتوي داخله سجلاً آخر هو Date.

كيفية الوصول إلى الحقول

عند وجود سجل داخل سجل آخر يتم الوصول إلى الحقول باستخدام النقطة (.) بشكل متسلسل:

s1.birthday.day	للوصول إلى يوم الميلاد:
s1.birthday.month	للوصول إلى الشهر:
s1.birthday.year	للوصول إلى السنة:

كلما ازداد عدد المستويات داخل السجلات نستخدم النقطة عدة مرات للوصول إلى الحقل المطلوب.

توضيح الفكرة

يمثل المتحول s1 سجلاً كاملاً للطالب، ويحتوي على عدة حقول مثل الاسم والرقم الجامعي. كما يحتوي أيضاً على سجل آخر هو birthday، والذي يضم بدوره الحقول day و month و year.

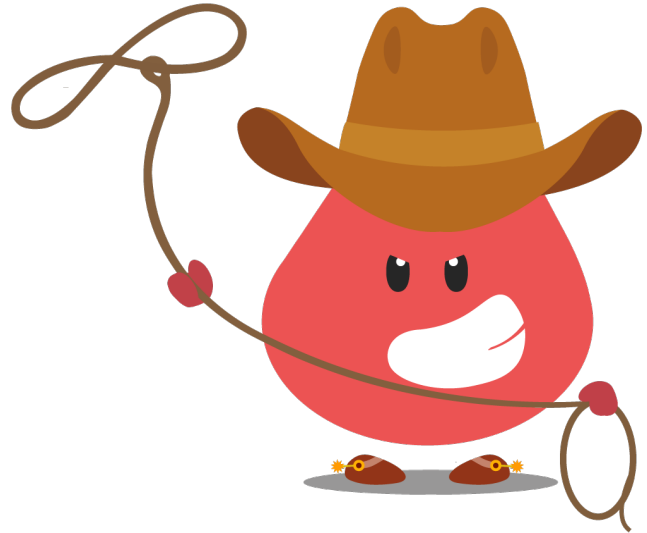
أهمية استخدام سجل ضمن سجل آخر

يمكننا تنظيم البيانات المرتبطة ببعضها ضمن سجلات متداخلة، مما يجعل الكود أكثر وضوحاً وتنظيماً.

مصفوفة داخل سجل (Struct داخل Array)

يمكن أن يحتوي السجل في C++ على مصفوفة كأحد حقوله، وذلك عندما نحتاج أن نخزن عدة قيم مرتبطة بنفس الكيان. مثال:

```
#include <iostream>
using namespace std;
struct Date {
    int day, month, year;
};
struct Student {
    string name;
    int ID;
    Date birthday;
    int marks[10];
};
int main() {
    Student s1;
    cin >> s1.name;
    cin >> s1.birthday.day;
    cin >> s1.birthday.month;
    cin >> s1.birthday.year;
    for (int i = 0; i < 10; i++)
        cin >> s1.marks[i];
    return 0;
}
```



في المثال السابق قمنا بتعريف مصفوفة باسم marks داخل سجل الطالب، وذلك لتخزين علامات الطالب في عدة مواد. عند إدخال القيم نستخدم حلقة تكرار لإدخال عناصر المصفوفة عنصراً عنصراً.

كيفية الوصول إلى عناصر المصفوفة داخل السجل

يتم الوصول إلى عناصر المصفوفة داخل السجل باستخدام النقطة ثم الفهرس:

s1.marks[0]	للوصول إلى أول علامة:
s1.marks[i]	للوصول إلى علامة معينة:

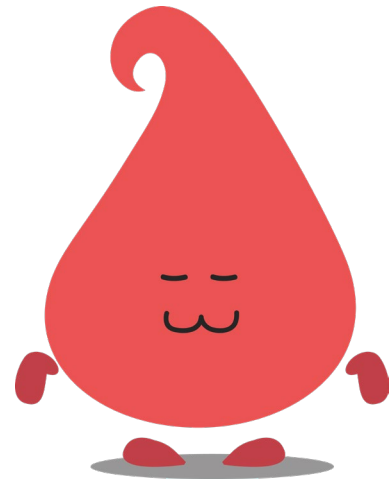
أهمية استخدام مصفوفة داخل سجل

يسمح استخدام مصفوفة داخل struct بتجميع بيانات متعددة مرتبطة بنفس الكيان ضمن متحول واحد. بدلاً من تعريف عدة متحولات منفصلة لعلامات الطالب يمكن جمعها داخل مصفوفة واحدة ضمن سجل الطالب، مما يجعل الكود أكثر تنظيماً وسهولة في التعامل.

مصفوفة من السجلات (Array of Struct)

عند الحاجة إلى تخزين بيانات بعدد كبير من الكيانات المتشابهة — مثل مجموعة من الطلاب — يمكننا تعريف مصفوفة من نوع struct بحيث يمثل كل عنصر في المصفوفة سجلاً كاملاً. فبدلاً من تعريف متحول واحد من نوع الطالب، يمكننا تعريف مصفوفة تحتوي عدة طلاب. مثال: لدينا 100 طالب في الجامعة

```
#include <iostream>
using namespace std;
struct Date {
    int day, month, year;
};
struct Student {
    string name;
    int ID;
    Date birthday;
    int marks[10];
};
int main() {
    Student arr[100];
    arr[0].birthday.year = 2000;
    arr[0].marks[0] = 80;
    arr[0].marks[1] = 70;
    arr[1].marks[0] = 88;
    return 0;
}
```



شرح المثال:

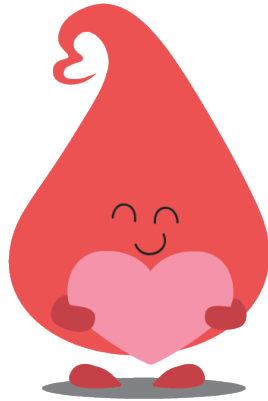
عندما كتبنا Student arr[100] فقد قمنا بتعريف مصفوفة تحتوي على 100 عنصر، وكل عنصر فيها يمثل طالباً كاملاً من النوع Student أي أن arr[0] يمثل الطالب الأول و arr[1] يمثل الطالب الثاني وهكذا... كل عنصر في هذه المصفوفة هو سجل كامل يحتوي الاسم والرقم الجامعي وتاريخ الميلاد والعلامات

كيفية الوصول إلى الحقول

للوصول إلى بيانات طالب معين داخل المصفوفة نكتب اسم المصفوفة ثم الفهرس ثم الحقل المطلوب.

arr[0].birthday.year = 2007;	تعيين سنة ميلاد الطالب الأول:
arr[0].marks[0] = 80;	إسناد علامة للمادة الأولى للطالب الأول:
arr[0].marks[1] = 70;	إسناد علامة أخرى للطالب الأول:
arr[1].marks[0] = 88;	إسناد علامة للطالب الثاني:

عند تعريف مصفوفة من نوع struct فإن كل عنصر في المصفوفة يمثل سجلاً كاملاً مستقلاً. وبذلك نستطيع التعامل مع عدد كبير من الكيانات المتشابهة بطريقة منظمة وسهلة. فبدل تعريف متحولات منفصلة لكل طالب، يمكن جمع جميع الطلاب داخل مصفوفة واحدة، ثم الوصول إلى بيانات كل طالب باستخدام رقم الفهرس الخاص به.



انتهى مقرر البرمجة 1 لا تنسونا من صالح دعائكم
نراكم في البرمجة 2 إن شاء الله ☺